IR_LCD

scannerizza la keyboard e manda il dato corrispondente via IR UART, ricevi sulla stessa board via IR UART il dato e trasmetti il simbolo corrispondente sullo schermo LCD.

Task	Periferiche
Trasmissione IR	• Led IR (PB10) • Timer 2 (PWM 38kHz)
Ricezione IR	• USART1 • Timer 3 (Interrupt 2400 Hz)
Scan Keyboard	• Timer 4 (Interrupt 500 Hz)
Display LCD	• Pin LCD

File .ioc

Pin

Funzione	Pin	Configurazione
LED IR (Tx)	PB10	TIM2_CH3
IR (Rx)	PA10	USART1_RX
Riga Keyboard	PC2	GPIO_Input
Riga Keyboard	PC3	GPIO_Input
Riga Keyboard	PC12	GPIO_Input
Riga Keyboard	PC13	GPIO_Input
Colonna Keyboard	PC8	GPIO_Output
Colonna Keyboard	PC9	GPIO_Output
Colonna Keyboard	PC10	GPIO_Output
Colonna Keyboard	PC11	GPIO_Output
LCD_BL_ON	PA4	GPIO_Output
LCD_E	PB1	GPIO_Output
LCD_RS	PB2	GPIO_Output
LCD_D4:D7	PB12:PB15	GPIO_Output

Timers

TIMx	Ruolo	Modalità	Frequenza	Priority
TIM2	Modulare la luce IR (Tx)	PWM	38 kHz	/

TIM3	Bit Rate (Tx)	Global Interrupt	2400 Hz	0
TIM4	Scansionare la Keyboard	Global Interrupt	500 Hz	1

- **Timer 2 (modulare la luce IR Tx):**
  - Channel 3: PWM GENERATION CH3
  - PSC=220
  - ARR=9
  - Pulse=5

$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(9+1)\cdot(220+1)} = 38.01\ kHz$$

$$DC = \frac{CCR_x}{ARR+1} = \frac{5}{9+1} = 50\%$$

- **Timer 3 (bit rate):**
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled
  - PSC=349
  - ARR=99

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(99+1)(349+1)} = 2400\ Hz$$

- **Timer 4 (scansionare la tastiera):**
  - Clock Source: Internal Clock
  - TIM4 global interrupt: Enabled
  - PSC=8399
  - ARR=19

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(19+1)(8399+1)} = 500\ Hz$$

Connectivity

- **USART1:**
  - Mode: Asynchronous
  - Baud Rate: 2400
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - NVIC: USART1 global interrupt = Enabled

Codice

Private Includes

```
#include <stdio.h>
#include "PMDB16_LCD.h"
```

Private TypeDef

```
// Stati Tx IR
typedef enum {
    TX_STATE_IDLE  = 0,
    TX_STATE_START = 1,
    TX_STATE_DATA  = 2,
    TX_STATE_STOP  = 3
} TxState_t;

// Keyboard
typedef struct {
       char character;
       uint8_t status;
       uint8_t duration;
       uint8_t sent;
} button;

typedef struct {
    GPIO_TypeDef* port;
    uint16_t pin;
} PinMap;
```

Private Define

```
// Keyboard Rows/Columns
#define KEY_C0 GPIOC,GPIO_PIN_11
#define KEY_C1 GPIOC,GPIO_PIN_10
#define KEY_C2 GPIOC,GPIO_PIN_9
#define KEY_C3 GPIOC,GPIO_PIN_8
#define KEY_R0 GPIOC,GPIO_PIN_3
#define KEY_R1 GPIOC,GPIO_PIN_2
#define KEY_R2 GPIOC,GPIO_PIN_13
#define KEY_R3 GPIOC,GPIO_PIN_12

// Keyboard Dimension
#define DIM 4

// Keyboard Buttons
#define DEBOUNCE 3
#define MAX_DURATION 250
```

Private Variables

```
// KEYBOARD
PinMap columns[] = {
    { KEY_C3 },
    { KEY_C2 },
    { KEY_C1 },
    { KEY_C0 }
};

PinMap rows[] = {
    { KEY_R0 },
    { KEY_R1 },
    { KEY_R2 },
```

```
    { KEY_R3 }
};

volatile button keyboard[DIM][DIM] = {
    { {'C', 1, 0, 0}, {'D', 1, 0, 0}, {'E', 1, 0, 0}, {'F', 1, 0, 0} },
    { {'8', 1, 0, 0}, {'9', 1, 0, 0}, {'A', 1, 0, 0}, {'B', 1, 0, 0} },
    { {'4', 1, 0, 0}, {'5', 1, 0, 0}, {'6', 1, 0, 0}, {'7', 1, 0, 0} },
    { {'0', 1, 0, 0}, {'1', 1, 0, 0}, {'2', 1, 0, 0}, {'3', 1, 0, 0} }
};

uint8_t active_col = 0;



// TX IR
volatile uint8_t  tx_data_buffer = 'G';
volatile uint8_t  bit_counter = 0;
volatile TxState_t tx_state = TX_STATE_IDLE;

// RX IR
uint8_t rx_data_buffer; // Byte ricevuto dall'IR

// LCD
volatile uint8_t ir_received_char = 0;
volatile uint8_t ir_data_ready = 0;
char lcd_buffer[17]; // Buffer per la stringa (16 char + terminatore)
```

Funzioni

```
// Avvia l'invio di un singolo byte tramite IR UART
void IR_UART_SendByte(uint8_t data)
{
    // Solo se il canale TX è libero posso iniziare nuova trasmissione(IDLE)
    if (tx_state == TX_STATE_IDLE)
    {
        tx_data_buffer = data;
        tx_state = TX_STATE_START;      // Ci prepariamo a inviare lo Start Bit

        __HAL_TIM_SET_COUNTER(&htim3, 0); // Azzera CNT di TIM3 affinchè lo Start Bit duri un
bit time pieno
        // e non di meno perchè magari CNT era ad un altro valore. CNT arriva a ARR (99) prima
di andare in
        // overflow e scattare l'ISR, se dalla scorsa ricezione CNT era ad un altro valore non
conta fino a
        // 100 ma di meno
        HAL_TIM_Base_Start_IT(&htim3); // TIM3 (2400 Hz) scandisce i bit nella Trasmissione IR
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) // 2400 Hz, UART1 (RX IR)
    {
        switch (tx_state)
        {
            case TX_STATE_IDLE:
              // Non dobbiamo inviare nulla, fermiamo il LED IR
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                break;
```

```
            case TX_STATE_START:
              // Abbiamo inviato lo Start Bit '0'
              // Ora passiamo ai dati
                HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // IR LED ON
                tx_state = TX_STATE_DATA; // Passa alla gestione dati
                bit_counter = 0;          // Resetta contatore bit
                break;

            case TX_STATE_DATA:
              // Inviamo 8 bit
              // Bit Masking
              // Esempio '1': 0(1)00 0001  >> 6  =  0000 000(1)
              // 0000 000(1) & 0000 0001 = 0000 0001 ON
              // Esempio '1': (0)100 0001  >> 7  =  0000 000(0)
                            // 0000 000(0) & 0000 0001 = 0000 0000 OFF

                if (((tx_data_buffer >> bit_counter) & 0x01) == 0)
                {
                    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // Bit '0' -> ON
                }
                else
                {
                    HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);  // Bit '1' -> OFF
                }

                bit_counter++; // Avanziamo nel buffer

                if (bit_counter >= 8) // Quando abbiamo finito
                {
                    tx_state = TX_STATE_STOP;
                }
                break;

            case TX_STATE_STOP: // Stop Bit ('1')
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3); // IR LED OFF

                // Ritorniamo in IDLE
                tx_state = TX_STATE_IDLE;
                // Finchè non premiamo altri tasti non abbiamo bisogno della Callback a 2400Hz
                HAL_TIM_Base_Stop_IT(&htim3);
                break;
        }
    }

    if (htim->Instance == TIM4) { // 500 Hz (scansione colonne KEYBOARD)
                for(int r=0; r<DIM; r++){
                      // Se il tasto è premuto
                    if(HAL_GPIO_ReadPin(rows[r].port, rows[r].pin) == GPIO_PIN_RESET){
                      // Se è premuto da poco
                        if(keyboard[r][active_col].duration < MAX_DURATION ) { // MAX_DURATION =
250*2ms = 0.5s
                            keyboard[r][active_col].duration++; // Aggiorniamo
                        }
                    } else { // Se il tasto non è premuto azzeriamo tutto
                        keyboard[r][active_col].duration = 0;
                        keyboard[r][active_col].sent = 0;
                    }
                }
                // Deselezioniamo la colonna appena letta
                HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin,
GPIO_PIN_RESET);
```

```
                // Aggiorniamo la colonna da attivare
                active_col=(active_col+1) % DIM;

                // Attiviamo la nuova colonna
                HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin,
GPIO_PIN_SET);
        }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
       ir_received_char = rx_data_buffer;
       ir_data_ready = 1; // Siamo pronti a scrivere sull'LCD
        HAL_UART_Receive_IT(&huart1, &rx_data_buffer, 1); // ci rimettiamo in ascolto
    }

}
```

Main

```
int main(void)
{

  // LCD initialization
  lcd_initialize();
  lcd_backlight_ON();

  // Prepara alla ricezione la UART1
  HAL_UART_Receive_IT(&huart1, &rx_data_buffer, 1);

  // Prepara la scansione della keyboard
  HAL_TIM_Base_Start_IT(&htim4);
  HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin, GPIO_PIN_SET);


  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
         // --- KEYBOARD SCANNING ---
                for(int j=0;j<DIM;j++){
                            for(int i=0;i<DIM;i++){

if((keyboard[i][j].duration>=DEBOUNCE)&&(keyboard[i][j].sent==0) ){
                                                IR_UART_SendByte(keyboard[i][j].character);
                                                keyboard[i][j].sent=1;
                                        }
                                }
                }


                // --- PRINT LCD  ---
                     if (ir_data_ready == 1) {

                            // Formatta la stringa col dato ricevuto
                            sprintf(lcd_buffer, "Character: %c", ir_received_char);
```

```
                    // Stampa
                    lcd_println(lcd_buffer, 0);

                    // Resetta la flag
                    ir_data_ready = 0;
                }
  }

}
```

IR_LED

scannerizza la keyboard e manda il dato corrispondente via IR UART, ricevi sulla stessa board via IR UART il dato e trasmetti il simbolo corrispondente sulla matrice Led.

Task	Periferiche
Trasmissione IR	• Led IR (PB10) • Timer 2 (PWM 38kHz)
Ricezione IR	• USART1 • Timer 3 (Interrupt 2400 Hz)
Scan Keyboard	• Timer 4 (Interrupt 500 Hz)
Matrice Led	• Pin Matrice Led • Timer 5 • SPI1

File .ioc

Pin

Funzione	Pin	Configurazione
LED IR (Tx)	PB10	TIM2_CH3
IR (Rx)	PA10	USART1_RX
Riga Keyboard	PC2	GPIO_Input
Riga Keyboard	PC3	GPIO_Input
Riga Keyboard	PC12	GPIO_Input
Riga Keyboard	PC13	GPIO_Input

Colonna Keyboard	PC8	GPIO_Output
Colonna Keyboard	PC9	GPIO_Output
Colonna Keyboard	PC10	GPIO_Output
Colonna Keyboard	PC11	GPIO_Output
SCK	PA5	SPI1
MISO	PA6	SPI1
MOSI	PA7	SPI1
RCLK	PB6	GPIO_Output

Timers

TIMx	Ruolo	Modalità	Frequenza	Priority
TIM2	Modulare la luce IR (Tx)	PWM	38 kHz	/
TIM3	Bit Rate (Tx)	Global Interrupt	2400 Hz	0
TIM4	Scansionare la Keyboard	Global Interrupt	500 Hz	1
TIM5	Refreshare colonne Led	Global Interrupt	250 Hz	2

- **Timer 2 (modulare la luce IR Tx):**
  - Channel 3: PWM GENERATION CH3
  - PSC=220
  - ARR=9
  - Pulse=5

$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(9+1)\cdot(220+1)} = 38.01\ kHz$$

$$DC = \frac{CCR_x}{ARR+1} = \frac{5}{9+1} = 50\%$$

- **Timer 3 (bit rate):**
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled
  - PSC=349
  - ARR=99

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(99+1)(349+1)} = 2400\ Hz$$

- **Timer 4 (scansionare la tastiera):**
  - Clock Source: Internal Clock

- TIM4 global interrupt: Enabled
- PSC=8399
- ARR=19

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(19+1)(8399+1)} = 500\ Hz$$

- **Timer 5 (refreshare le colonne della matrice LED)**:
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled (priority = 2)
  - PSC=99
  - ARR=3359

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(3359+1)(99+1)} = 250\ Hz$$

Connectivity

- **USART1:**
  - Mode: Asynchronous
  - Baud Rate: 2400
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - NVIC: USART1 global interrupt = Enabled

- SPI1:
  - Mode: Full Duplex Master
  - Data size: 8 bits
  - Prescaler (for Baud Rate): 4
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA2 stream3 global interrupt = Enabled
    - SPI1 global interrupt = Enabled

Codice

Private Include

```
#include "stdio.h"
#include "string.h"
```

Private TypeDef

```
// Stati Tx IR
typedef enum {
    TX_STATE_IDLE  = 0,
    TX_STATE_START = 1,
    TX_STATE_DATA  = 2,
    TX_STATE_STOP  = 3
} TxState_t;

// Keyboard
typedef struct {
       char character;
       uint8_t duration;
       uint8_t sent;
       uint8_t idx;
} button;

typedef struct {
    GPIO_TypeDef* port;
    uint16_t pin;
} PinMap;
/* USER CODE END PTD */
```

Private Define

```
// Keyboard Rows/Columns
#define KEY_C0 GPIOC,GPIO_PIN_11
#define KEY_C1 GPIOC,GPIO_PIN_10
#define KEY_C2 GPIOC,GPIO_PIN_9
#define KEY_C3 GPIOC,GPIO_PIN_8
#define KEY_R0 GPIOC,GPIO_PIN_3
#define KEY_R1 GPIOC,GPIO_PIN_2
#define KEY_R2 GPIOC,GPIO_PIN_13
#define KEY_R3 GPIOC,GPIO_PIN_12

// Keyboard Dimension
#define DIM 4

// Keyboard Buttons
#define DEBOUNCE 3
#define MAX_DURATION 250

// --- Matrice LED ---
#define NUM_COLS_LED 5
#define NUM_SYMBOLS 16
#define RCLK_PORT GPIOB
#define RCLK_PIN  GPIO_PIN_6
```

Private Variables

```
// KEYBOARD
PinMap columns[] = {
    { KEY_C3 },
    { KEY_C2 },
    { KEY_C1 },
```

```
    { KEY_C0 }
};

PinMap rows[] = {
    { KEY_R0 },
    { KEY_R1 },
    { KEY_R2 },
    { KEY_R3 }
};


volatile button keyboard[DIM][DIM] = {
    { {'C', 0, 0, 12}, {'D', 0, 0, 13}, {'E', 0, 0, 14}, {'F', 0, 0, 15} },
    { {'8', 0, 0, 8}, {'9', 0, 0, 9}, {'A', 0, 0, 10}, {'B', 0, 0, 11} },
    { {'4', 0, 0, 4}, {'5', 0, 0, 5}, {'6', 0, 0, 6}, {'7', 0, 0, 7} },
    { {'0', 0, 0, 0}, {'1', 0, 0, 1}, {'2', 0, 0, 2}, {'3', 0, 0, 3} }
};

uint8_t active_col = 0;



// TX IR
volatile uint8_t  tx_data_buffer = 'G';
volatile uint8_t  bit_counter = 0;
volatile TxState_t tx_state = TX_STATE_IDLE;

// RX IR
uint8_t rx_data_buffer; // Byte ricevuto dall'IR
volatile uint8_t ir_received_char = 0;
volatile uint8_t ir_data_ready = 0;

// Matrice Led
static const uint8_t COL_LED[NUM_COLS_LED] = {16, 8, 4, 2, 1};
static const uint8_t SYMBOLS[NUM_SYMBOLS+1][NUM_COLS_LED] = {
    { 62,  69,  73,  81,  62}, // '0'
    {  0,  33, 127,   1,   0}, // '1'
    { 33,  67,  69,  73,  49}, // '2'
    { 34,  65,  73,  73,  54}, // '3'
    { 12,  20,  36, 127,   4}, // '4'
    {114,  81,  81,  81,  78}, // '5'
    { 30,  41,  73,  73,   6}, // '6'
    { 64,  71,  72,  80,  96}, // '7'
    { 54,  73,  73,  73,  54}, // '8'
    { 48,  73,  73,  74,  60}, // '9'
    { 63,  72,  72,  72,  63}, // 'A'
    {127,  73,  73,  73,  54}, // 'B'
    { 62,  65,  65,  65,  34}, // 'C'
    {127,  65,  65,  65,  62}, // 'D'
    {127,  73,  73,  73,  65}, // 'E'
    {127,  72,  72,  72,  64}, // 'F'
    {  0,   0,   0,   0,   0}  // spento
};
static volatile uint8_t sym_idx = NUM_SYMBOLS;   // cosa mostra la matrice (spenta se
NUM_SYMBOLS)
static uint8_t col_cnt = 0;               // colonna della matrice in refresh
```

Funzioni

```
static inline void spi_send(uint8_t col, uint8_t row){
      static uint8_t buffer[2]; // 2 byte
```

```
	// DAISY RING
	buffer[0]=row; // prima la riga
	buffer[1]=col; // poi la colonna

	// Ogni 4 ms trasmettiamo (refreshamo)
	HAL_SPI_Transmit_DMA(&hspi1, buffer, 2);
}

void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi){
	if(hspi->Instance==SPI1){
		// Una volta terminata la comunicazione dei 2 byte
		// diamo un impulso su RCLK per far scattare il latch
		// dei due shift register
		HAL_GPIO_WritePin(RCLK_PORT, RCLK_PIN, GPIO_PIN_SET);
		HAL_GPIO_WritePin(RCLK_PORT, RCLK_PIN, GPIO_PIN_RESET);
	}
}


// Avvia l'invio di un singolo byte tramite IR UART
void IR_UART_SendByte(uint8_t data)
{
    // Solo se il canale TX è libero posso iniziare nuova trasmissione(IDLE)
    if (tx_state == TX_STATE_IDLE)
    {
        tx_data_buffer = data;
        tx_state = TX_STATE_START;      // Ci prepariamo a inviare lo Start Bit

        __HAL_TIM_SET_COUNTER(&htim3, 0); // Azzera CNT di TIM3 affinchè lo Start Bit
duri un bit time pieno
        // e non di meno perchè magari CNT era ad un altro valore. CNT arriva a ARR
(99) prima di andare in
        // overflow e scattare l'ISR, se dalla scorsa ricezione CNT era ad un altro
valore non conta fino a
        // 100 ma di meno
        HAL_TIM_Base_Start_IT(&htim3); // TIM3 (2400 Hz) scandisce i bit nella
Trasmissione IR
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) // 2400 Hz, UART1 (RX IR)
    {
        switch (tx_state)
        {
            case TX_STATE_IDLE:
             // Non dobbiamo inviare nulla, fermiamo il LED IR
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                break;

            case TX_STATE_START:
             // Abbiamo inviato lo Start Bit '0'
             // Ora passiamo ai dati
                HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // IR LED ON
                tx_state = TX_STATE_DATA; // Passa alla gestione dati
                bit_counter = 0;          // Resetta contatore bit
                break;

            case TX_STATE_DATA:
             // Inviamo 8 bit
```

```
            // Bit Masking
            // Esempio '1': 0(1)00 0001  >> 6  =  0000 000(1)
            // 0000 000(1) & 0000 0001 = 0000 0001 ON
            // Esempio '1': (0)100 0001  >> 7  =  0000 000(0)
                        // 0000 000(0) & 0000 0001 = 0000 0000 OFF

                if (((tx_data_buffer >> bit_counter) & 0x01) == 0)
                {
                    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // Bit '0' -> ON
                }
                else
                {
                    HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);  // Bit '1' -> OFF
                }

                bit_counter++; // Avanziamo nel buffer

                if (bit_counter >= 8) // Quando abbiamo finito
                {
                    tx_state = TX_STATE_STOP;
                }
                break;

            case TX_STATE_STOP: // Stop Bit ('1')
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3); // IR LED OFF

                // Ritorniamo in IDLE
                tx_state = TX_STATE_IDLE;
                // Finchè non premiamo altri tasti non abbiamo bisogno della Callback a
2400Hz
                HAL_TIM_Base_Stop_IT(&htim3);
                break;
        }
    }

    if (htim->Instance == TIM4) { // 500 Hz (scansione colonne KEYBOARD)
              for(int r=0; r<DIM; r++){
                   // Se il tasto è premuto
                  if(HAL_GPIO_ReadPin(rows[r].port, rows[r].pin) == GPIO_PIN_RESET){
                   // Se è premuto da poco
                      if(keyboard[r][active_col].duration < MAX_DURATION ) {
                          keyboard[r][active_col].duration++; // Aggiorniamo
                      }
                  } else { // Se il tasto non è premuto azzeriamo tutto
                      keyboard[r][active_col].duration = 0;
                      keyboard[r][active_col].sent = 0;
                  }
              }
              // Deselezioniamo la colonna appena letta
              HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin,
GPIO_PIN_RESET);

              // Aggiorniamo la colonna da attivare
              active_col=(active_col+1) % DIM;

              // Attiviamo la nuova colonna
              HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin,
GPIO_PIN_SET);
       }
```

```
    if (htim->Instance == TIM5) { // 4ms, refresh colonne
      spi_send(COL_LED[col_cnt], SYMBOLS[sym_idx][col_cnt]);
      col_cnt=(col_cnt+1)%NUM_COLS_LED;
    }



}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
      ir_received_char = rx_data_buffer;
      ir_data_ready = 1; // Siamo pronti a scrivere sulla matrice Led
       HAL_UART_Receive_IT(&huart1, &rx_data_buffer, 1); // ci rimettiamo in ascolto
    }

}

/* Logica ASCII nel while
 * static uint8_t char_to_idx(uint8_t c)
{
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    return NUM_SYMBOLS;   // carattere sconosciuto -> matrice spenta
}
 */
```

Main

```
  // Prepara alla ricezione la UART1
  HAL_UART_Receive_IT(&huart1, &rx_data_buffer, 1);

  // Prepara la scansione della keyboard
  HAL_TIM_Base_Start_IT(&htim4);
  HAL_GPIO_WritePin(columns[active_col].port, columns[active_col].pin, GPIO_PIN_SET);

  // Matrice Led
  HAL_TIM_Base_Start_IT(&htim5);

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
        // --- KEYBOARD SCANNING ---
              for(int j=0;j<DIM;j++){
                          for(int i=0;i<DIM;i++){

if((keyboard[i][j].duration>=DEBOUNCE)&&(keyboard[i][j].sent==0) ){
                                        IR_UART_SendByte(keyboard[i][j].character);
                                        keyboard[i][j].sent=1;
                                }
                          }
              }
```

```
                // MATRICE LED
                if (ir_data_ready==1){
                      // Cerca l'idx del simbolo da mostrare

                      // Tramite i for
                      for (int j=0; j<DIM; j++){
                            for (int i=0; i<DIM; i++){
                                  if(ir_received_char == keyboard[i][j].character){
                                        sym_idx=keyboard[i][j].idx;
                                  }
                            }
                      }
                      ir_data_ready = 0;

                      /* o tramite la logica ASCII
                       * if (ir_data_ready) {
                                 sym_idx = char_to_idx(ir_received_char);
                                 ir_data_ready = 0;
                           }
                       */
                }


  }
  /* USER CODE END 3 */
}
```